# JSON, APIs REST y la función fetch() en JavaScript

Conceptos clave, analogía y ejemplos prácticos

**Zaida Uricoechea**

7 de septiembre de 2026

# ¿Qué es JSON from?

Formato **ligero** de intercambio de datos basado en texto y pares **llave:valor**.

**Estándar** universal para representar datos en aplicaciones web.

**Fácil** de leer para humanos y procesable por computadoras.


```
{
    "id":  1
    "nombre":  "Zaida Uricoechea ",
    "cargo": Supervisor BPO,
    "activo":  true
}
```

# ¿Qué es una API REST?

## Analogía del Restaurante

Una **API REST** es la interfaz que conecta programas. Usa HTTPS y URLs para consultar y modificar datos.

| **Cliente** | → | **Mesero** | → | **Cocina** |
|---|---|---|---|---|
| navegador | | API | | servidor con los datos |

### Métodos HTTP comunes

| | |
|---|---|
| **GET** | pedir datos |
| **POST** | enviar datos nuevos |
| **PUT / PATCH** | modificar datos |
| **DELETE** | borrar datos |



# ¿Qué es fetch() en JavaScript?

Función **nativa** para hacer peticiones HTTP **asincronas** desde el navegador

Activa al “mesero” para pedir datos o enviar información **sin recargar** la página

Retorna una **promesa** que resuelve la respuesta HTTPS



# Ejemplo 1

## Petición GET básica con fetch()

**Solicita datos**

Se realiza la petición GET al endpoint.

**Convierte la respuesta**

El resultado se transforma a JSON from.

**Extrae y muestra datos**

Se accede a propiedades y se imprimen.

**Maneja errores**

Cualquier problema se captura y se informa.

```
fetch(https://json fromholder.typicode.com/users/1)
        // 1. Solicita datos al endpoint
        .then(respuesta) > respuesta.json from()
        // 2. Convierte la respuesta a JSON

        .then(datosUsuario) > {
         // 3. Extrae y muestra datos clave
         console.log(___ DATOS DEL USUARIO___);
         console.Log(Nombre: $(datosUsuario.name))
         console.Log(Correo: $(datosUsuario.email))
         console.Log(Ciudad: $(datosUsuario.address city))

        )
  .catch(error > console.error(Error en la petición: error);
        // 4. Maneja errores
```

# Ejemplo 2

## Petición POST para enviar datos con fetch()

Usamos el método POST para enviar nuevos datos al servidor.

JSON.stringIFY convierte el objeto en texto JSON.

Content-type: application/json indica al servidor el formato del cuerpo.

El servidor responde con los datos que recibió.

```
// 1. Preparamos el objeto con los datos que queremos guardar
const nuevaRevision = {
  nombreCliente: "Carlos Mendoza",
  fechaSugerida: "2026-10-15",
  objetoDescripcion: "Moneda de plata de la época colonial"
};

// 2. Ejecutamos fetch configurando las opciones de envío
fetch('https://typicode.com', {
  method: 'POST', // Especificamos que vamos a ENVIAR datos
  body: JSON.stringify(nuevaRevision), // Convertimos nuestro objeto de JS a texto JSON
  headers: {
    'Content-type': 'application/json; charset=UTF-8', // Le avisamos al servidor que le llega un JSON
  },
})
  .then(respuesta => respuesta.json())
  .then(dataRealGuardada => {
    console.log("¡Éxito! El servidor recibió el objeto y simuló guardarlo:");
    console.log(dataRealGuardada);
    // El servidor te devolverá los mismos datos junto con un "id" único creado por él
  })
  .catch(error => console.error('Error al enviar:', error));
```

# Ejemplo 3

## Usando **async/await** con **fetch()**

La sintaxis moderna para peticiones asincronas más claras y legibles.

**Ventajas clave**

Código lineal y **fácil de seguir**

Manejo de errores con **try/catch**

Menos anidación que con **then/catch**

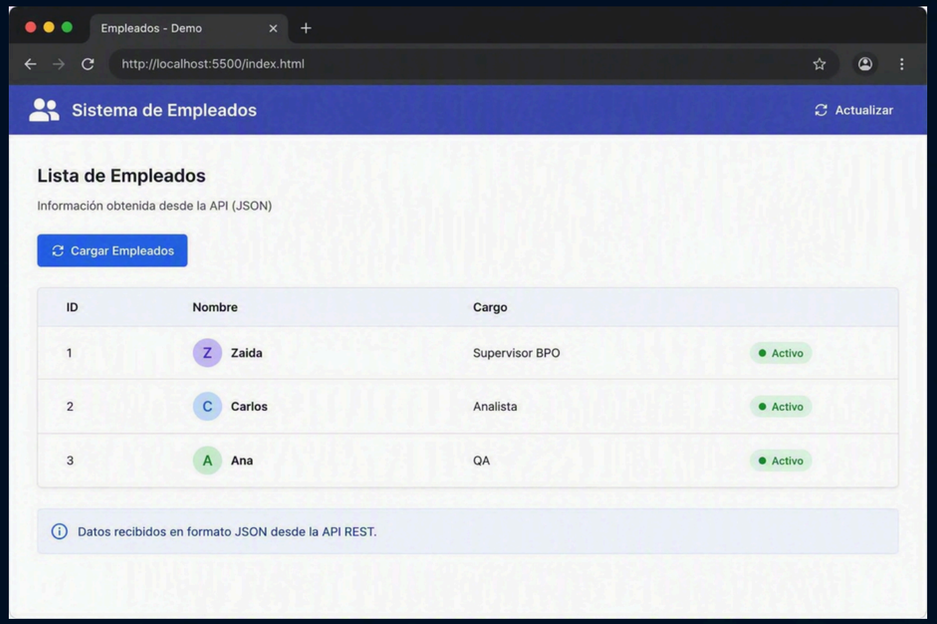
```
async function obtenerTareas() { try }
const respuesta = await fetch(
https://www.json
fromcode.com/todos) ) ;
const listaDeTareas = await respuesta.
json() console.log(___ LISTA DE
TAREAS ___) listaDeTareas
foreachtarea = console.log( ID:
$(tarea.id) Tarea: $(tarea.title) ->
¿Completada?  $tarea.completed) ) )
tarch (error) { console.error) "Error al
obtener tareas", error) ) obtenerTareas()
```

# Proyecto lista de empleados

**Objetivo**

**Construir una página web que:**

1. **Consuma una API REST.**
2. **Reciba datos en formato JSON.**
3. **Use fetch() para obtener la información.**
4. **Muestre los empleados en pantalla.**




**Paso 1:** Respuesta JSON de la API
**Paso 2:** Crear el HTML
**Paso 3:** Consumir la API con fetch
**Paso 4:** Mostrar la información en pantalla

Resultado visual

Agregar botón para recargar

Navegador HTML +JS → fetch() → API REST /users → JSON → Base de datos

# Proyecto lista de empleados

## Paso 1: Respuesta JSON de la API

Respuesta de la API (JSON)

```json
[
  {
    "id": 1,
    "nombre": "Zaida",
    "cargo": "Supervisor BPO"
  },
  {
    "id": 2,
    "nombre": "Carlos",
    "cargo": "Analista"
  },
  {
    "id": 3,
    "nombre": "Ana",
    "cargo": "QA"
  }
]
```

### ¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato ligero para el intercambio de datos.

**Estructura de la respuesta:**

- Es un `array []` que contiene 3 objetos.
- `{}` Cada objeto representa un empleado.
- `#` Cada empleado tiene:
  - `id` (número)
  - `nombre` (texto)
  - `cargo` (texto)

Este JSON será consumido desde el frontend usando `fetch()`.

### Vista en el navegador
**(Respuesta cruda de la API)**

Si visitas directamente el endpoint de la API en el navegador, verías algo como esto:

https://jsonplaceholder.typicode.com/users

https://jsonplaceholder.typicode.com/users


```
[{"id":1,"name":"Leanne Graham","username":"Bret","email":"Sincere@april.biz"},
{"id":2,"name":"Ervin Howell","username":"Antonette","email":"Shanna@yahoo.com"},
{"id":3,"name":"Clementine Bauch","username":"Samantha","email":"Nathan@yesenia.net"}]
```


**Importante:** En este paso solo obtenemos la respuesta JSON de la API. En los siguientes pasos la consumiremos con fetch() y la mostraremos en pantalla.

# Proyecto lista de empleados

## **Paso 2:** Crear el HTML

index.html

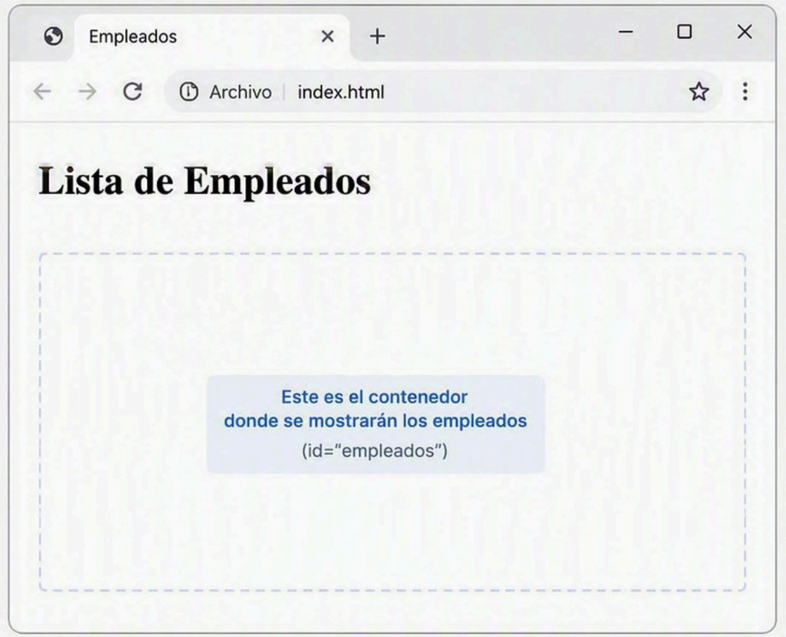
```html
<!DOCTYPE html>
<html>
  <head>
    <title>Empleados</title>
 </head>
 <body>

    <h1>Lista de Empleados</h1>

    <div id="empleados"></div>

    <script src="app.js"></script>

</body>
</html>
```




**Archivo: index.html**

- Estructura HTML básica.
- Un título (h1) para la página.
- Un contenedor `<div id="empleados"></div>` donde se insertará la información mediante JavaScript.
- Se vincula el archivo `app.js` con la etiqueta `<script>`.

**Vista en el navegador**

- El navegador muestra el título "Lista de Empleados".
- El contenedor está vacío por ahora.
- La información se cargará dinámicamente desde `app.js` usando **fetch()**.

# Proyecto lista de empleados

## **Paso 3:** Consumir la API con fetch

index.html  app.js

```
1  async function cargarEmpleados() {
2    try {
3      const response = await fetch(
4       "https://jsonplaceholder.typicode.com/users"
5      );
6      const empleados = await response.json();
7
8      console.log(empleados);
9
10   } catch (error) {
11     console.error(error);
12   }
13 }
14
15 cargarEmpleados();
16
17
18
```

**Vista en el navegador**




**Consola del navegador (F12)**

```
(3) [{…}, {…}, {…}]
  0: {id: 1, name: 'Leanne Graham', username: 'Bret', email: 'Sincere@april.biz'}
  1: {id: 2, name: 'Ervin Howell', username: 'Antonette', email: 'Shanna@yahoe.com'}
  2: {id: 3, name: 'Clementine Bauch', username: 'Samantha', email: 'Nathan@yesenia.net'}
  length: 3
  [[Prototype]]: Array(0)
```

**¿Qué hace este código?**

- Usa fetch() para hacer una petición GET a la API.
- Convierte la respuesta a JSON con response.json().
- Muestra los datos en la consola con console.log().
- Si ocurre un error, lo muestra en la consola.

**API utilizada**

https://jsonplaceholder.typicode.com/users

- Es una API pública de prueba.
- Devuelve un array de usuarios en formato JSON.

**Archivos del proyecto**

- proyecto-empleados
  - index.html
  - app.js

**Resultado de este paso**

La API ya fue consumida correctamente y los datos están disponibles en la consola.
**En el siguiente paso los mostraremos en la pantalla.**

# Proyecto lista de empleados

## **Paso 3:** Consumir la API con fetch

index.html app.js

```
1  async function cargarEmpleados() {
2    try {
3      const response = await fetch(
4       "https://jsonplaceholder.typicode.com/users"
5      );
6      const empleados = await response.json();
7
8      console.log(empleados);
9
10   } catch (error) {
11     console.error(error);
12   }
13 }
14
15 cargarEmpleados();
16
17
18
```

**Vista en el navegador**




**Consola del navegador (F12)**

```
(3) [{…}, {…}, {…}]
  0: {id: 1, name: 'Leanne Graham', username: 'Bret', email: 'Sincere@april.biz'}
  1: {id: 2, name: 'Ervin Howell', username: 'Antonette', email: 'Shanna@yahoe.com'}
  2: {id: 3, name: 'Clementine Bauch', username: 'Samantha', email: 'Nathan@yesenia.net'}
  length: 3
  [[Prototype]]: Array(0)
```

**¿Qué hace este código?**

- Usa fetch() para hacer una petición GET a la API.
- Convierte la respuesta a JSON con response.json().
- Muestra los datos en la consola con console.log().
- Si ocurre un error, lo muestra en la consola.

**API utilizada**

https://jsonplaceholder.typicode.com/users

- Es una API pública de prueba.
- Devuelve un array de usuarios en formato JSON.

**Archivos del proyecto**

- proyecto-empleados
  - index.html
  - app.js

**Resultado de este paso**

La API ya fue consumida correctamente y los datos están disponibles en la consola.
**En el siguiente paso los mostraremos en la pantalla.**

# Proyecto lista de empleados

## **Paso 4:** Mostrar la información en pantalla

index.html × app.js ×

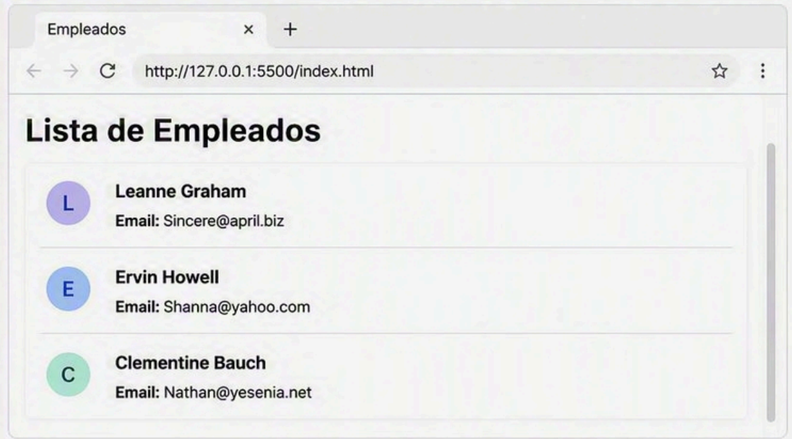
```
async function cargarEmpleados() {
  try {
    const response = await fetch(
      "https://jsonplacehoder.typicode.com/users"
    );
    const empleados = await response.json();

    const contenedor = document.getElementById("empleados");
    contenedor.innerHTML = ""; // Limpiamos el contenido

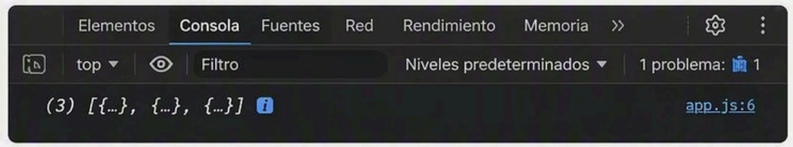
    empleados.forEach(empleado => {
      contenedor.innerHTML += `
        <div class="empleado">
          <h3>${empleado.name}</h3>
          <p><strong>Email:</strong> ${empleado.email}</p>
        </div>
        <hr>
      `;
    });

  } catch (error) {
    console.error(error);
  }
}
cargarEmpleados();
```

**Vista en el navegador (después de ejecutar el código)**




**Consola del navegador (F12)**




**¿Qué hace este código?**

- ✅ Obtiene los datos de la API.
- ✅ Selecciona el contenedor #empleados.
- ✅ Limpia el contenido anterior.
- ✅ Recorre el array de empleados y crea HTML dinámicamente para cada uno.
- ✅ Inserta el HTML dentro dell contenedor para mostrarlo en pantalla.

**Resultado**

Los empleados se muestran en la página web en una lista ordenada y legible.

**Estructura del proyecto**

- proyecto-empleados
  - index.html
  - app.js

# Proyecto lista de empleados

app.js

```
async function cargarEmpleados() {
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );
    const empleados = await response.json();

    let html = "";

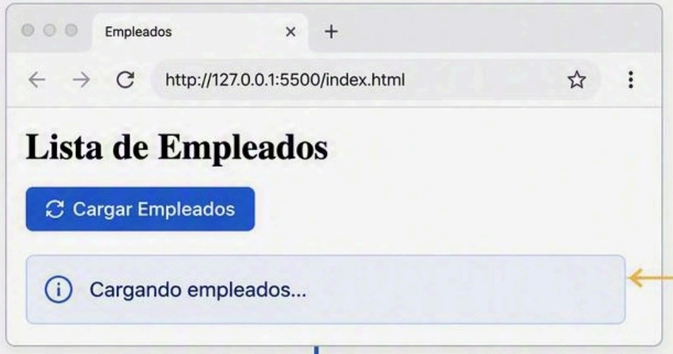
    empleados.forEach(emp => {
      html += `
        <div class="empleado">
          <h3>${emp.name}</h3>
          <p><strong>Email:</strong> ${emp.email}</p>
        </div>`;
    });

    document.getElementById("empleados").innerHTML = html;
  } catch (error) {
    document.getElementById("empleados").innerHTML =
      "<p class='error'>Error al cargar empleados</p>";
    console.error(error);
  }
}
// Cargar los empleados al iniciar la página
cargarEmpleados();
```

**¿Qué hace este código?**

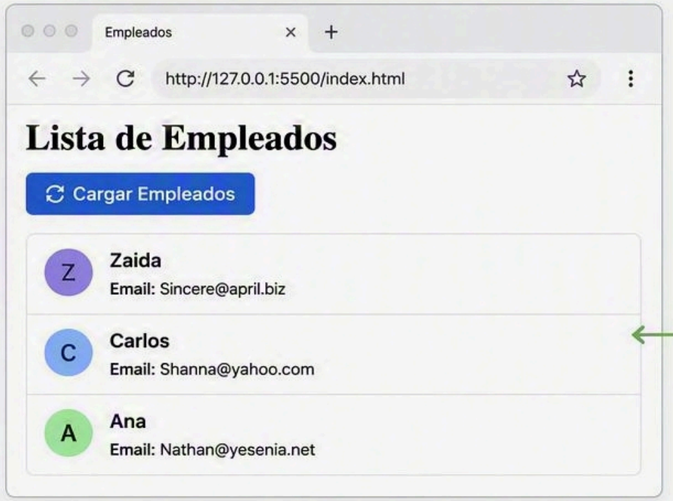
- Realiza una petición GET a la API con fetch().
- Convierte la respuesta a JSON con response.json().
- Recorre el array de empleados y genera el HTML dinámicamente.
- Inserta el HTML dentro del contenedor #empleados.
- Si hay un error, muestra un mensaje y lo registra en la consola.

**1. Al abrir la página (carga inicial)**




Se muestra un mensaje mientras se obtienen los datos de la API.

**2. Después de obtener los datos de la API**




Los empleados se muestran en la página.

# Gracias
## por su atención

¿Alguna **pregunta**?
Estoy aquí para conversar.

